TECHMART DATA WAREHOUSE

Apache Hop ETL Practical — Structured Step-by-Step Guide

Project scope: MySQL source database → Apache Hop transformations → MySQL warehouse tables

This guide documents the workflow completed in the conversation: Customers, Products, Orders, Payments and Deliveries. It includes the SQL setup, Hop pipeline structure, validation rules, execution checks and final verification.

Important scope note: The ETL pipelines and record-count verification were completed. OLAP analysis was discussed as a possible next stage, but no OLAP operations or data cube implementation have yet been confirmed as completed. Check your assignment brief to determine whether OLAP is required.

1. Project overview

The project moves operational data from the **techmart_source** database into warehouse tables in **techmart_dw**. Apache Hop is used to extract rows, clean selected fields, validate records and load valid or rejected records into separate destinations.

Area	What it does	Tool / location
Source	Original operational tables	MySQL: techmart_source
ETL	Extract, transform, validate and load	Apache Hop GUI
Warehouse	Stores cleaned/validated data	MySQL: techmart_dw

Source tables used

customers, products, orders, payments, deliveries

Main Hop components used

Table Input, String Operations, If Null, Filter Rows and Table Output. The Customers workflow also used a Formula transform and Select Values to create and retain a standardized city field.

General pipeline pattern

Table Input → **optional cleaning transforms** → **optional Filter Rows** → **Table Output**. Where a validation split is used, TRUE and FALSE paths go to separate Table Output transforms.

2. Environment and database setup

Environment recorded during the project

• macOS laptop
• Apache Hop GUI
• MySQL running locally on localhost:3306
• Hop database connection: **TechMart_Source**
• Source database: **techmart_source**
• Warehouse database: **techmart_dw**

Connection configuration

In Hop, create/use a database connection with host **localhost**, port **3306**, database **techmart_source**, username **root** (as configured in this project). The Table Output transforms use the same connection but specify target schema **techmart_dw**.

Folder and save-path note

Save each .hpl pipeline inside the TechMart project folder. If Hop reports that a parent directory does not exist, check the exact path spelling. In the encountered error, Hop's path contained **projecsts** while Finder showed **projects**. Use Save As and select the existing folder rather than creating duplicate project folders.

3. Warehouse tables

The following SQL represents the target tables used in the workflow. Run the statements in MySQL if you need to recreate the warehouse tables. If a table already exists, do not rerun CREATE TABLE blindly; inspect it first.

Customers

```
CREATE TABLE techmart_dw.stg_customers (
  customer_id INT, customer_name VARCHAR(100),
  email VARCHAR(100), city VARCHAR(50),
  state VARCHAR(50), registration_date DATE
);
```

```
CREATE TABLE techmart_dw.dim_customer (
  customer_id INT PRIMARY KEY,
  customer_name VARCHAR(100), email VARCHAR(100),
  city VARCHAR(50), state VARCHAR(50),
  registration_date DATE
);
```

Products

```
CREATE TABLE techmart_dw.stg_products (
  product_id INT, product_name VARCHAR(100),
  category VARCHAR(50), price DECIMAL(10,2),
  stock_quantity INT
);

CREATE TABLE techmart_dw.dim_product (
  product_id INT PRIMARY KEY,
  product_name VARCHAR(100), category VARCHAR(50),
  price DECIMAL(10,2), stock_quantity INT
);

CREATE TABLE techmart_dw.rejected_products LIKE techmart_dw.stg_products;
```

Orders

```
CREATE TABLE techmart_dw.fact_orders (
  order_id INT PRIMARY KEY, customer_id INT,
  product_id INT, quantity INT, order_date DATETIME,
  order_status VARCHAR(30)
);

CREATE TABLE techmart_dw.rejected_orders (
  order_id INT, customer_id INT, product_id INT,
  quantity INT, order_date DATETIME,
  order_status VARCHAR(30)
);
```

Payments

```
CREATE TABLE techmart_dw.fact_payments (
  payment_id INT PRIMARY KEY, order_id INT,
  payment_method VARCHAR(50), payment_amount DECIMAL(10,2),
  payment_status VARCHAR(30), payment_date DATETIME
);

CREATE TABLE techmart_dw.rejected_payments (
  payment_id INT, order_id INT, payment_method VARCHAR(50),
  payment_amount DECIMAL(10,2), payment_status VARCHAR(30),
  payment_date DATETIME
);
```

Deliveries

```
CREATE TABLE techmart_dw.fact_deliveries (
  delivery_id INT PRIMARY KEY, order_id INT,
  delivery_address VARCHAR(255), delivery_city VARCHAR(50),
  delivery_status VARCHAR(30), dispatch_date DATETIME,
  delivery_date DATETIME
);

CREATE TABLE techmart_dw.rejected_deliveries (
  delivery_id INT, order_id INT, delivery_address VARCHAR(255),
  delivery_city VARCHAR(50), delivery_status VARCHAR(30),
  dispatch_date DATETIME, delivery_date DATETIME
);
```

4. Apache Hop pipeline procedures

General steps for each pipeline

1. Create a new pipeline in the TechMart project.
2. Add the required transforms to the canvas.
3. Connect transforms using hops (Shift-drag between transforms, as used in this project).
4. Double-click each transform and configure it.
5. For Table Input, select **TechMart_Source** and enter the SQL query.
6. For Table Output, select **TechMart_Source**, set target schema to **techmart_dw**, set the target table, and click **Get fields** to verify mappings.
7. Save the .hpl file, run it, inspect the execution log and verify the target tables in MySQL.

4.1 Customers — two-pipeline workflow

Pipeline 1: 01_Extract_Customers.hpl

Structure: Table Input → Table Output. Read the customers source table and write 10 records to techmart_dw.stg_customers.

```
SELECT customer_id, customer_name, email, city, state, registration_date
FROM customers;
```

Pipeline 2: 02_Transform_Customers.hpl

Structure: Table Input → String Operations → If Null → Formula → Select Values → Table Output.

- Table Input reads from techmart_dw.stg_customers.
- String Operations: trim city on both sides and convert to lowercase.
- If Null: replace NULL city with **Unknown**.
- Formula: create **clean_city** using **PROPER([city])** (String result).
- Select Values: rename clean_city to city and remove the original city field.
- Table Output: load to techmart_dw.dim_customer.

Result verified: 10 rows in dim_customer. Examples of standardization included pUNE → Pune, MUMBAI → Mumbai and NULL → Unknown.

4.2 Products — 03_Products_ETL.hpl

Structure: Table Input → String Operations → If Null → Filter Rows → two Table Outputs.

```
SELECT product_id, product_name, category, price, stock_quantity
FROM products;
```

Transform configuration

- String Operations: category → trim both sides, lowercase.
- If Null: replace NULL category with **Unknown**.
- Filter Rows condition: **stock_quantity >= 0**.
- TRUE → Load_valid → techmart_dw.dim_product.
- FALSE → Load_invalid → techmart_dw.rejected_products.
- Both outputs use TechMart_Source, target schema techmart_dw, and all five fields mapped.

Expected and verified result: 11 valid products and 1 rejected product. The rejected row was Smartwatch (product_id 10) with stock_quantity -5. The duplicate Wireless Mouse names with different product IDs were retained; they were not treated as invalid.

Rerun caution: Table Output appends rows. Before rerunning a full-load pipeline, clear its target tables if appropriate, for example TRUNCATE TABLE techmart_dw.dim_product; and TRUNCATE TABLE techmart_dw.rejected_products;. Do this only when you intend to replace the existing load.

4.3 Orders — 04_Orders_ETL.hpl

Structure: Table Input → Filter Rows → two Table Outputs.

```
SELECT order_id, customer_id, product_id, quantity, order_date, order_status
FROM orders;
```

Transform configuration

• Filter Rows condition: **quantity >= 1**.
• TRUE → Load_valid → techmart_dw.fact_orders.
• FALSE → Load_invalid → techmart_dw.rejected_orders.
• Map all six fields: order_id, customer_id, product_id, quantity, order_date, order_status.

Expected and verified result: 19 valid orders and 1 rejected order. The rejected row was order_id 17 with quantity -1. Cancelled/Pending statuses were not rejected solely because of their status.

4.4 Payments — Payments ETL pipeline

Structure: Table Input → Filter Rows → two Table Outputs. The pipeline filename was not explicitly confirmed in the conversation; use the naming convention you saved in Hop.

```
SELECT payment_id, order_id, payment_method, payment_amount,
       payment_status, payment_date
FROM payments;
```

Transform configuration

• Filter Rows condition: **payment_amount >= 0**.
• TRUE → Load_valid → techmart_dw.fact_payments.
• FALSE → Load_invalid → techmart_dw.rejected_payments.
• Map all six fields: payment_id, order_id, payment_method, payment_amount, payment_status, payment_date.

Expected and verified result: 9 valid payments and 1 rejected payment with a negative payment_amount. Pending and Failed payment statuses were retained; they were not treated as invalid solely because of status.

4.5 Deliveries — 05_Delivery.hpl

Structure: Table Input → String Operations → If Null → Table Output. No Filter Rows split was used.

```
SELECT delivery_id, order_id, delivery_address, delivery_city,
       delivery_status, dispatch_date, delivery_date
FROM deliveries;
```

Transform configuration

• String Operations on delivery_city: trim both sides and lowercase.
• If Null: replace NULL delivery_city with **Unknown**.
• Do not replace NULL dispatch_date or delivery_date; these can represent pending, cancelled or not-yet-delivered orders.
• Table Output → techmart_dw.fact_deliveries; map all seven fields.

Expected and verified result: 10 records loaded into fact_deliveries. The project workflow did not route deliveries to rejected_deliveries.

5. Final verification in MySQL

Run this query to verify the row counts across the warehouse tables:

```
SELECT 'Customers' AS entity, COUNT(*) AS records
FROM techmart_dw.dim_customer
UNION ALL
SELECT 'Products', COUNT(*) FROM techmart_dw.dim_product
UNION ALL
SELECT 'Rejected Products', COUNT(*) FROM techmart_dw.rejected_products
UNION ALL
SELECT 'Orders', COUNT(*) FROM techmart_dw.fact_orders
UNION ALL
SELECT 'Rejected Orders', COUNT(*) FROM techmart_dw.rejected_orders
UNION ALL
SELECT 'Payments', COUNT(*) FROM techmart_dw.fact_payments
UNION ALL
SELECT 'Rejected Payments', COUNT(*) FROM techmart_dw.rejected_payments
UNION ALL
SELECT 'Deliveries', COUNT(*) FROM techmart_dw.fact_deliveries;
```

Actual counts reported and verified by the user

Warehouse table	Records
dim_customer	10
dim_product	11
rejected_products	1
fact_orders	19
rejected_orders	1
fact_payments	9
rejected_payments	1
fact_deliveries	10
Total valid	59
Total rejected	3
Total source records accounted for	62

Useful spot checks

```
SELECT * FROM techmart_dw.rejected_products;
SELECT * FROM techmart_dw.rejected_orders;
SELECT * FROM techmart_dw.rejected_payments;
SELECT * FROM techmart_dw.fact_deliveries;
```

The rejected tables should contain the invalid rows described above. Deliveries should contain 10 rows, including valid NULL dispatch/delivery dates where applicable.

6. OLAP — what is and is not completed

OLAP (Online Analytical Processing) is an analysis stage that uses warehouse data to answer business questions. It is separate from ETL. The conversation established and verified the ETL outputs, but did not establish that OLAP operations were performed. If the assignment requires OLAP, treat this as an additional task.

Typical OLAP operations

Roll-up: aggregate data to a higher level, e.g. city → state.
Drill-down: move from summary to detail, e.g. category → product.
Slice: select one dimension value, e.g. one city.
Dice: select a subset across multiple dimensions, e.g. selected cities and categories.

Pivot: rotate dimensions to view the same data from another perspective.

Example analytical query (not proof that OLAP was required or completed)

```
SELECT p.category,
       SUM(o.quantity * p.price) AS estimated_order_value
FROM techmart_dw.fact_orders o
JOIN techmart_dw.dim_product p
  ON o.product_id = p.product_id
GROUP BY p.category;
```

This example estimates order value using the current product price and order quantity. It is not a reconciled revenue measure and does not use the payments table. A proper sales/revenue metric may need business rules for cancellations, returns, discounts and payment status.

7. Practical and viva checklist

Before submission

- ■ Save all .hpl pipelines in the TechMart project folder.
- ■ Confirm each pipeline runs without errors.
- ■ Keep the source database and warehouse database intact.
- ■ Capture screenshots of each pipeline canvas and successful execution log.
- ■ Capture the final MySQL count query output.
- ■ Confirm from the assignment brief whether OLAP, reports, charts or a data cube are additionally required.

Questions to prepare for viva

1. What is ETL and why is it needed?
2. What is the difference between a source database and a data warehouse?
3. What does Table Input do?
4. What is the purpose of String Operations and If Null?
5. How does Filter Rows route TRUE and FALSE records?
6. Why are rejected records stored separately?
7. Why were NULL delivery dates preserved?
8. What happens if a Table Output pipeline is run twice?
9. What is the difference between ETL and OLAP?
10. What is roll-up, drill-down, slice, dice and pivot?

Rerun and duplicate-key warning

The project encountered duplicate rows and a duplicate primary-key error when pipelines were rerun without clearing their target tables. Table Output appends by default. For a repeatable full load, clear the relevant target tables before rerunning, or configure an appropriate upsert/merge strategy if your assignment requires incremental loading. Never truncate source tables.

End of guide — TechMart ETL workflow